

弹性且经济高效的作业调度 异构数据中心:一种软约束 方法

徐恒文中南大学商学院
中国长沙

hengwenxu@csu.edu.cn

郝寅*清华大学

数字生产与智能物流研究中心,北京,中国 yinh24@mails.tsinghua.edu.cn

摘要 传统的刚性调度模型在高峰负载或资源稀缺的情况下往往难以保持可行性,这主要是由于物理和网格约束的僵化。为了克服这一问题,我们提出了一种适用于异构数据中心的弹性调度框架,该框架将这些硬性边界重新构建到一个软约束的混合整数线性规划 (MILP)模型中。我们通过引入三类新的有效割集来解决这一NP难问题的计算难题:启动有效性、快速循环预防和最小停机机前瞻,这些割集有效地收紧了线性松弛。

实验表明,与标准基线相比,这些削减措施可将大规模实例的求解时间减少60%以上。此外,敏感性分析证实了该模型的弹性,即使电力可用性下降到峰值需求的50%,也能确保运行连续性和平稳降级。

索引词 异构数据中心、弹性调度、软约束优化、混合整数线性规划 (MILP)、有效割集、优雅降级。

一、引言及相关工作

在现代数据中心中,如何在运营效率和物理基础设施限制之间取得平衡是一项根本性的挑战。随着数据中心在全球电力需求中所占份额不断增长[1],运营商面临着越来越大的成本削减压力。然而,将基础设施纯粹视为抽象概念的方法往往忽略了动态运行造成的机械磨损和电气性能下降[2]。

节能调度已得到广泛研究。动态整合[3]和电力价格套利[4]、[5]的基础研究表明,节能调度能够显著降低成本。

然而,这些理想化的模型往往忽略了状态转换带来的开销。Lin等人[6]指出,服务器循环会产生巨大的成本 既包括能源延迟,也包括硬件损耗 这可能会抵消动态电源管理带来的节能效果。

电网交互增加了另一层复杂性。为了确保稳定性,电力公司对功率爬坡施加了严格的限制,这些限制通常在需求响应文献中被建模为刚性的“硬”约束[7]–[9]。在容易出现流量突发的生产环境中,这种刚性边界往往会导致调度问题在数学上不可行,从而在可用性最关键的时候导致求解器失效。

一个稳健的调度器必须优先考虑通过可控降级而非系统故障来保证可行性。虽然软约束在实时操作系统中已成为标准做法[10],但它们在整体数据中心调度中的应用仍然有限[11][12]。为了弥补这一差距,我们提出了一种基于惩罚优化模型的弹性调度框架。通过将服务水平协议 (SLA)、电力容量、爬坡速率和碳排放等约束重新表述为软约束,我们的模型即使在极端负载下也能保持可行性。此外,为了解决该模型的计算复杂度问题,我们推导出了有效的割集,以缩小搜索空间并提高可解性。

二、配方

A. 问题描述 我们致力于解决异

构数据中心在严格的物理和网格约束下进行的联合调度和资源管理问题。核心挑战在于如何将批处理作业分配给具有不同功耗和损耗特性的服务器,同时还要满足动态爬坡和碳排放限制。为了确保在高峰负载下问题的可解性,我们将容量、截止时间 and 负载均衡方面的硬性限制放宽为软性限制。

我们的目标是通过平衡运营支出 (包括电力和硬件损耗)与违反约束的加权处罚,来最大限度地降低总体成本,从而确保系统弹性。

B. 数学模型

表 I 总结了模型中使用的集合、参数和决策变量。弹性调度问题的混合整数线性规划 (MILP) 公式如下所示:

$$\begin{aligned} \min Z = & \sum_{k \in K} \sum_{t \in T} \rho_k \frac{P_k}{C_k} + \beta \sum_{k \in K} \sum_{t \in T} \text{wear}_{k,t} + c \sum_{k \in K} \sum_{t \in T} \text{start}_{k,t} \\ & + \sum_{j \in J} \rho_j \text{SLA}_j + \sum_{t \in T} \text{penalty}_t \\ & + \sum_{t \in T} \rho_{\text{ramp}} \text{ramp}_t + \rho_{\text{carb}} \text{carb}_t \end{aligned}$$

表一	
术语和参数定义	
符号定义	
集合与索引	
S,k	异构服务器集合,索引 k ∈ {1, . . . , n}
J,j,i	批处理作业集合,索引 j, i ∈ {1, . . . , m}
T,t	离散时间段集合,索引 t ∈ {1, . . . , T }
作业和工作负载参数	
DJ,k	服务器 k 上作业 j 的处理时间
DLj	Job j 的软性截止日期
系统和物理参数空间	
P _k	服务器 k 的功耗 (空闲/开启)
P _k ^峰	服务器 k (满载)的功耗
和	t 时刻的碳排放强度
C _{max} ^(max)	最大总功率容量
C _{max}	最大允许碳排放量上限carb ΔPramp最大允许功率爬升率
T _分 ^近	最短连续开启时间
T _分	最短连续关闭时间
Δbal	负载不平衡的目标阈值
成本与惩罚系数	
t _{elec}	t 时刻随时间变化的电价
wear _c	基础硬件损耗成本系数
开始c	单服务器启动成本
β _k	服务器 k 的磨损敏感度权重
未SLA粉	每次违反截止日期的处罚
斜坡	每单位电力容量违规的罚款
碳氢化合	每单位爬坡速率违规的罚款
物hw	每单位碳排放上限违规的罚款
平衡	违反最低开/关机频率规定的处罚
	每单位负荷不平衡违规行为的罚款
决策变量	
stj,k,t xj,k,t	二进制:1,如果作业 j 在 t 时刻在 k 上启动;二进制:1,
uk,t zk,t 总	如果作业 j 在 t 时刻在 k 上执行;二进制:1,如果服务器 k
计	在 t 时刻处于开启状态;二进制:1,如果服务器 k 在
	t 时刻启动;服务器 k 在 t 时刻的总功率;违规松弛变量
P _{k,t}	(例如 v) k,k'
在 ...	巴尔

$$(t + Dj,k - 1)stj,k,t \leq DLj + v_j, \forall j$$
$$k \in S, t \in T \tag{8}$$

$$P_{k,t}^{全部的} \leq C_{k,t}^{最大爬升} + v_t^{力量}, \quad \forall t \in T$$
$$k \in S \tag{9}$$

$$(P_{k,t}^{总计} - P_{k,t-1}^{全部的}) \leq \Delta Pramp + v_t^{爬坡}, \quad \forall t \geq 2$$
$$k \in S \tag{10}$$

$$(P_{k,t-1}^{全部的} - P_{k,t}^{全部的}) \leq \Delta Pramp + v_t^{爬坡}, \quad \forall t \geq 2$$
$$k \in S \tag{11}$$

$$z_{k,\tau} \leq u_{k,t} + v_{k,t}, \quad \forall k, t$$
$$\tau = t - T_{min} + 1$$
$$u_{k,t-1} - u_{k,t} \leq 1 - u_{k,\tau} + v_{k,t},$$

$$\forall k, \forall \tau \in \{t + 1, \dots, \min(t + T_{min}, |T|)\} \tag{12}$$
$$P_{k,t}^{全部的} \leq C_{k,t}^{最大爬升} + v_{k,t}^{碳水化}$$
$$t \in T, k \in S \tag{13}$$

$$u_{k',t} - u_{k,t} \leq \Delta bal + v_{k,k'}, \forall k, k' \in S$$
$$t \in T \tag{14}$$

$$u_{k',t} - u_{k,t} \leq \Delta bal + v_{k,k'}, \forall k, k' \in S$$
$$t \in T \tag{15}$$

$$st, x, u, z \in \{0, 1\}; v \dots \geq 0 \tag{16}$$

$$st, x, u, z \in \{0, 1\}; v \dots \geq 0 \tag{17}$$

目标函数 (1) 旨在最小化广义总成本,该成本包括电力消耗、硬件损耗、启动惩罚以及软约束违反的加权惩罚。关于物理操作,约束 (2) 至 (6) 定义了功耗、确保作业分配的唯一性,并将执行限制在活动服务器上,而公式 (7) 则控制启动逻辑。

我们在式 (8) 至 (16) 中构建了软约束。式 (8) 允许在受到惩罚的情况下违反截止止时间。为了确保电网稳定性和硬件寿命,约束 (9) 至 (13) 调节了功率容量、爬坡速率和最小开/关持续时间。最后,式 (14) 和 (15) 至 (16) 考虑了环境和负载均衡目标,允许在可控范围内进行偏差,以保持可行性。

三、算法设计

A. 分支切割算法框架

我们使用分支切割 (B&C)算法[13]求解第二节中提出的NP难混合整数线性规划 (MILP)模型。根据标准的整数规划原则[14],我们将二元变量 (例如, xj,k,t, zk,t)的整数约束放宽为连续区间[0, 1]。我们采用最佳边界搜索策略进行节点选择,以优先选择有希望的区域。

在此框架下,分离过程动态生成当前分数阶解所违反的有效割,以收紧松弛。如果没有找到违反的割,我们应用最分数阶分支策略,选择……

x

总空闲 k uk,t + (P st P = P k,t

总速峰值 k P k

j ∈ J

∀ k, t

stj,k,t = 1, ∀ j ∈ J

k ∈ S, t = 1

t

xj,k,t ≤ 1, ∀ k ∈ S, t ∈ T

j ∈ J

xj,k,t ≤ uk,t, ∀ j, k, t uk,t - uk,t-1 ≤

zk,t, ∀ k, t ≥ 2

算法 1分支切割算法框架	
需要: MILP 模型参数 (作业数 J、服务器数 S、时间 T)	
确保:最佳进度安排 S	目标值 Z
1:初始化:全局上界 $UB \leftarrow -\infty$,下界 $LB \leftarrow -\infty$	
2:使用 LP 松弛创建根节点N0 ; $L \leftarrow \{N0\}$	
3:当 $L = \emptyset$ 时	
4:使用最佳边界策略选择节点 $N_k \in L$; $L \leftarrow L \setminus \{N_k\}$	
5:重复	
6:	求解 N_k 的 LP 松弛问题以获得 x_k^* 和价值 ZLP
7:	如果LP 不可行或 $ZLP \geq UB$,则
8:	修剪节点 N_k ;中断
9:	如果结束
10:	如果 x_k^* 是整数
11:	如果 $ZLP < UB$,则
12:	$UB \leftarrow ZLP$;S 结束 if x_k^*
13:	
14:	按最优性进行剪枝 ;打破
15:	如果结束
16:	分离程序 :检查方程 (18)是否违反, (19) 、 (20)
17:	如果发现违规切割,则
18:	对 N_k 添加切割 ;继续 (重新求解线性规划)
19:	别的
20:	中断 (继续分支)
21:	如果结束
22:	直到节点被修剪或准备好进行分支
23:	如果 N_k 没有被剪枝,那么
24:	选择分数变量 v (优先于 x_j, k, t)
25:	生成子代 $N0_k (v = 0)$, $N1 (v = 1)$;添加到 L
26:	结束 if
27:	结束 while 循环
28:	返回S , UB

具有最高整数不可分割性的变量
搜索空间。

B. 有效削减

为了加速收敛,我们引入了三类
明确禁止物理或逻辑上无效的有效切割
分数阶线性规划解中经常出现状态转换。

$$z_k, t \leq u_k, t, \forall k \in S, t \in T \text{ (18)}$$
$$z_k, t + z_k, t+1 \leq 1, \forall k \in S, t \in \{1, \dots, |T| - 1\} \text{ (19)}$$
$$u_k, t-1 - u_k, t + u_k, t+1 \leq 1, \forall k, t \in \{2, \dots, |T| - 1\} \text{ (20)}$$

我们通过引入三类来加强LP松弛。
有效切割。公式 18 中的启动有效性切割强制执行
通过将启动成本与运营成本联系起来,保持逻辑一致性
状态。此约束确保启动成本 z_k, t 不会
分配给一个非活动服务器,其中 $u_k, t = 0$ 。接下来,

公式 19 中的快速循环预防削减涉及数值
通过消除分数 “微循环”来消除异常情况。它明确禁止在连续时间段内启动。

这是标准放松练习中经常被忽视的状况。最后,
公式 20 中的最小向下前瞻截断可防止危险情况的发生。
例如快速的开-关-开序列等振荡模式。
不平等加剧了最小停机时间限制。
分离循环;具体来说,如果服务器从
在 $t - 1$ 时刻开启,在 t 时刻关闭,则强制保持关闭状态。
 $t + 1$,实际上强制规定了至少 2 个时隙的停机时间。
算法 1 详细描述了整个过程。

四、数值实验

A. 实验设置和实例

我们在一台配备 AMD 处理器的工作站上进行了实验。
Ryzen 7 5800H CPU (3.20 GHz)和 32 GB 内存。
所有算法均使用 Python 3.10.5 实现并求解。
使用 Gurobi Optimizer 11.0.1 的 MILP 模型存在 MIP 间隙
容差为0.01%。为了验证不同条件下的性能
为了应对复杂性,我们生成了一个包含 15 个实例的数据集,这些
实例被分为小、中、大三种规模。工作负载
反映了现实世界的异构性,其中服务器具有不同的峰值
怠速功率 $P_k^{\text{空闲}}$ 峰值功率 $P_k^{\text{峰值}}$ 特征。
此外,我们还在作业处理中引入了随机噪声。
倍数 D_j, k 用于模拟建筑亲和力。动态环境因素,例如电价 $c_t^{\text{电子}}$ 和碳
强度等,纳入真实的峰谷波动
测试模型的时序调度能力。

表2
测试实例的尺度定义

规模	服务器 (S)	工作 (J)	时间范围 (T)	数量
小的	3	15	10	5
中等的	5	24	12	5
大的	10	40	20	5

B. 参数设置和对比方案

广义总成本综合考虑了运营支出和软约束违规的罚款。我们进行了校准。

惩罚系数 ρ 显著超过标准
运营成本,确保求解器优先考虑物理成本
可行性,并且仅在资源极其丰富的情况下放宽限制。
竞争。为了模拟资源稀缺,我们限制了……
最大功率容量 C 达到峰值需求的70%。我们以此为基准进行评估。
两种求解器配置 :基础模型 ,代表
标准MILP公式和Base+Cuts模型,
包含了公式 19、18 和 20 中定义的有效切割
收紧线性松弛。求解器时间限制已设置。
小号、中号和大号分别需要 20 秒、100 秒和 600 秒。
分别是实例。

C. 实验结果与分析

表3详细列出了两者的计算性能。
标准公式和所提出的截断增强模型,

重点关注求解时间和探索的分支切割节点数。这是对所有十五个问题中的一个基本观察结果。

例如,两种配置均成功收敛。

达到全局最优解,MIP 差距为 0.00%。

这种一致性证实了引入有效的削减措施是合理的。

严格收紧线性松弛空间,而不进行任何修剪

可行的整数解。因此,主要指标

评估指标包括搜索空间(节点)的减少量和

由此产生的算法效率(时间)。

表3
测试中所有配置的详细实验结果
实例

尺度识别方法	节点	时间 (秒)
小的	1 基数 27,433 基础值+削减 10,741	3.3 1.8
	2 基准 51,487 基础价格+扣除额 19,640	6.2 3.2
	3 基数 8,424 基础值+削减 6,198	2.3 1.5
	4 基数 96,131 基础值+削减 32,191	5.8 2.1
	5 根据 7,541 基础款+切割 756	1.3 0.4
	1 基数 49,362 基础值+削减 17,030	30.5 13.0
	2 基数 107,799 基础值+扣除额 26,346	41.4 16.3
	3 基数 8,587 基础值+削减 4,352	11.8 4.6
	4 基数 7,614 基础值+削减 6,757	9.9 5.5
	5 基数 17,757 基础值+扣除额 10,539	16.0 10.1
中等的	1 基准 20,526 基础值+扣除额 17,921	509.1 313.1
	2 基数 11,345 基础价格+折扣 4,279	482.2 182.3
	3 基准 55,165 基础值+扣除额 83,898	451.6 387.8
	4 基数 111,872 基础值+削减 65,673	403.2 222.9
	5 基准 6,635 基础价格+折扣 8,599	524.9 515.2
大的	1 基准 20,526 基础值+扣除额 17,921	509.1 313.1
	2 基数 11,345 基础价格+折扣 4,279	482.2 182.3
	3 基准 55,165 基础值+扣除额 83,898	451.6 387.8
	4 基数 111,872 基础值+削减 65,673	403.2 222.9
	5 基准 6,635 基础价格+折扣 8,599	524.9 515.2
	1 基准 20,526 基础值+扣除额 17,921	509.1 313.1
	2 基数 11,345 基础价格+折扣 4,279	482.2 182.3
	3 基准 55,165 基础值+扣除额 83,898	451.6 387.8
	4 基数 111,872 基础值+削减 65,673	403.2 222.9
	5 基准 6,635 基础价格+折扣 8,599	524.9 515.2

注意:所有实例均达到最优解 (OPT)。

节点计数分析揭示了其背后的机制

性能提升。在中小规模

实例中,Base+Cuts 配置显著降低了

搜索树的大小。例如,在中等规模的搜索树中

数据集中,平均探索节点数减少了

约48%。这一减少表明有效

修剪有效地剔除了违反条件的分数阶解

物理逻辑(例如,快速循环或无效启动),从而

允许求解器更早地舍弃次优分支。

搜索过程。这种修剪功能直接转化为

根据观察到的运行时改进情况,Base+Cuts

与之前的配置相比,该配置实现了平均 42.7% 的速度提升。

适用于中等规模实例的标准模型。

在代表实际运行核心测试场景的大规模实验中,可扩展性优势更加显著。虽然标准基础模型接近

在多种情况下,600 秒的时间限制,基础+剪辑

配置展现出了相当强的韧性。值得注意的是,在

实例 2,节点数量下降了超过 62% (从 11,345 个减少到 11,345 个)。

至 4,279),从而促进溶液的相应减少。

时间从 482.2 秒缩短到 182.3 秒。即使在以下情况下:

节点数量减少的幅度较小,甚至略有逆转。

由于启发式方法的变化(例如,实例 3),求解器仍然

运行时间更短,表明删减部分有所改进。

下限的质量并指导了搜索方向

更高效地找到最优解。这些结果验证了这一点。

拟议削减措施的协同效应:通过明确

禁止LP弛豫过程中出现不合逻辑的状态转换,

该框架实现了关键的平衡,最大限度地减少了

在确保运营可行性的前提下,总体成本

计算开销显著降低。

五.敏感性分析

为了评估所提出方法的可扩展性和鲁棒性

在 Base+Cuts 配置下,我们进行了全面的敏感性分析,改变了问题规模和资源约束。实验采用的基准设置是 $|S| = 5$,

$|J| = 24$, $|T| = 12$,容量比 $\eta = 0.7$,平均老化结果取自三次独立运行,每次运行时间为300秒。

超时。结果如图 1 所示,显示出不同的结果。

不同维度上的行为。

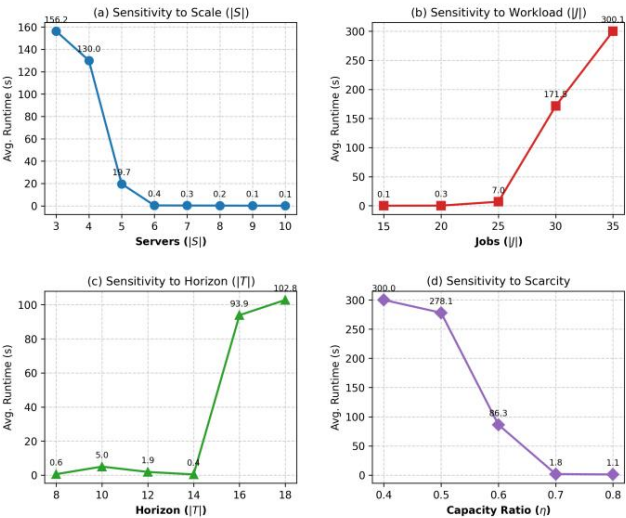


图 1. 四个维度上求解器运行时间的敏感性分析: (a) 服务器数量 $|S|$, (b)作业数量 $|J|$, (c)时间范围 $|T|$,以及 (d) 容量比 η 。求解器时间限制设置为300秒。

资源可用性与计算复杂度之间存在显著的负相关关系。

在服务器维度上。如图 1(a) 所示,受限

基础设施 ($|S| = 3$)迫使求解器在高度复杂的环境中运行

搜索空间受限,导致运行时间超过 150 分钟。

几秒钟。但是,增加服务器数量可以缓解这些限制。

可行性边界使得分支切割算法能够在 $|S| \geq 6$ 时几乎瞬间 (<0.5 秒)识别出低成本调度方案。相反,作业维度 (图1(b))决定了主要的计算瓶颈。对于最多25个作业的工作负载,系统表现出稳定的性能;超过25个作业后,赋值变量的组合爆炸会导致运行时间急剧上升,并在 $|U|=35$ 时持续达到时间限制。时间范围 (图1(c))的影响则较为温和,复杂度在可控范围内增长,直到更长的规划周期 ($|T| \geq 16$)引入更深的搜索树。

至关重要,容量比分析 (图 1(d))突显了模型在压力下的韧性。在资源极度匮乏 ($\eta \leq 0.5$)的情况下,传统的刚性模型很可能返回不可行状态,而我们的软约束框架在 300 秒的时间窗口内持续优化惩罚权衡。这种高计算成本反映了在饱和系统中最小化违规行为的难度。随着资源恢复正常 ($\eta \geq 0.7$),软约束的压力减轻,求解器恢复到高效率状态。这证实了所提出的方法能够在临界负载条件下实现性能的平滑降级,同时在正常运行期间保持速度。

六、结论

本研究针对传统刚性调度模型固有的可行性挑战,提出了一种适用于异构数据中心的软约束混合整数线性规划 (MILP)框架。该模型将严格的物理和网络限制转化为基于惩罚的目标,即使在传统方法通常失效的峰值负载条件下,也能确保运行的连续性。

为了克服NP难调度问题的计算复杂度,我们推导并整合了三类有效割:启动有效性、快速循环预防和最小下降前瞻约束。数值实验表明,这些结构性改进显著收紧了线性松弛,在大规模实例中实现了超过60%的计算加速,同时保持了全局最优性。

此外,敏感性分析揭示了该方法的卓越韧性。尽管计算成本不可避免地会随着工作负载密度的增加而上升,但该系统在资源极度匮乏的情况下仍能成功实现优雅降级。调度器不会在电力容量不足时返回不可行的状态,而是优化服务水平和运行惩罚之间的权衡。这一能力证实,所提出的框架为面临能源供应不确定性和工作负载波动性日益加剧的现代数据中心提供了一种实用且可扩展的解决方案。

参考

[1] E. Masanet,A. Shehabi,N. Lei,S. Smith 和 J. Koomey,“重新校准全球数据中心能源使用估算”,《科学》,第 367 卷,第 6481 期,第 984-986 页,2020 年。
[2] R. Buyya,A. Beloglazov 和 J. Abawajy,“云计算数据中心资源的节能管理:愿景、架构要素和开放性挑战”,arXiv 预印本 arXiv:1006.0308,2010 年。

[3] A. Beloglazov,J. Abawajy 和 R. Buyya,“面向云计算数据中心高效管理的节能资源分配启发式算法”,未来一代计算机系统,第 28 卷,第 5 期,第 755-768 页,2012 年。
[4] Y. Zhang,Y. Wang 和 X. Wang,“Greenware:绿色云规模数据中心以最大限度地利用可再生能源”,载于 ACM/IFIP/USENIX 分布式系统平台和开放分布式处理国际会议。Springer,2011 年,第 143-164 页。
[5] L. Rao,X. Liu,L. Xie 和 W. Liu,“最小化电力成本:多电力市场环境分布式互联网数据中心的优化”,载于 2010 年 IEEE INFOCOM 会议论文集。IEEE,2010 年,第 1-9 页。
[6] M. Lin,A. Wierman,LL Andrew 和 E. Thereska,“功率比例数据中心的动态合理配置”,IEEE/ACM 网络事务,第 21 卷,第 5 期,第 1378-1391 页,2012 年。
[7] NH Tran,DH Tran,S. Ren,Z. Han,E.-N. Huh 和 CS Hong,“地理分布式数据中心如何进行需求响应:一种博弈论方法”,IEEE智能电网学报,第7 卷,第2期,第937-947页,2015年。
[8] H. Wang,J. Huang,X. Lin 和 H. Mohsenian-Rad,“数据中心的主动需求响应:双赢的解决方案”,IEEE 智能电网汇刊,第 7 卷,第 3 期,第 1584-1596 页,2015 年。
[9] I. Goiri,W. Katsak,K. Le,TD Nguyen 和 R. Bianchini,“Parasol and greenswitch:管理由可再生能源供电的数据中心”,ACM SIGPLAN Notices,第 48 卷,第 4 期,第 51-64 页,2013 年。
[10] G. Buttazzo,G. Lipari,L. Abeni 和 M. Caccamo,《软实时系统:可预测性与效率》。Springer,2005 年。
[11] Z. Xiao,W. Song 和 Q. Chen,“使用虚拟机在云计算环境中进行动态资源分配”,IEEE 并行与分布式系统汇刊,第 24 卷,第 6 期,第 1107-1117 页,2012 年。
[12] Z. Zhou, F. Liu, Y. Xu, R. Zou, H. Xu, JC Lui 和 H. Jin,“面向地理分布式云服务的碳感知负载均衡”,载于 2013 年 IEEE 第 21 届计算机和电信系统建模、分析和仿真国际研讨会论文集。IEEE,2013 年,第 232-241 页。
[13] M. Padberg 和 G. Rinaldi,“用于解决大规模对称旅行商问题的分支切割算法”,SIAM 评论,第 33 卷,第 1 期,第 60-100 页,1991 年。
[14] L.A. Wolsey,《整数规划》。John Wiley & Sons 出版社,2020 年。